

02 · Conversations in depth

Siddharth M. Bhatia · siddharth@sidmb.com

July 1, 2026

Contents

Build participants and a conversation manually	1
Framing ownership (who sees what)	2
Turn-taking	2
Stopping	3
Branching — fork a conversation for free	3
Fork from a specific turn — <code>branch_from</code>	4
Editing history in place — rewind, edit, reset	4
Editing history and the KV cache	5
Ephemeral sampling — read state without mutating it	5
Reasoning visibility (CoT models)	6

`Conversation.from_models` is a convenience wrapper. Build a `Conversation` by hand when you want **per-speaker framing** (different system prompts / private context), a custom moderator, policies, or hooks.

Build participants and a conversation manually

`AutoModelParticipant.from_pretrained(...)` is the HF-style loader (the participant analog of `AutoModelForCausalLM.from_pretrained`): it loads the model by id and returns the family-correct participant instance. Loading the same id twice shares one model object (weights are process-cached), so both speakers below share weights.

```
from interlens import Conversation, AutoModelParticipant

alice = AutoModelParticipant.from_pretrained(
    "Qwen/Qwen2.5-3B-Instruct", name="alice", device="cuda",
    system_prompt="You are a concise, skeptical debater. Keep
    replies under 3 sentences.",
    temperature=0.7, max_new_tokens=200,
)
bob = AutoModelParticipant.from_pretrained(
```

```

    "Qwen/Qwen2.5-3B-Instruct", name="bob", device="cuda",      #
    same id → shares alice's weights (cached)
    system_prompt="You are an enthusiastic optimist who loves
    analogies.",
    temperature=0.9, max_new_tokens=200,
)
conv = Conversation(
    participants=(alice, bob),
    shared_context="Topic: should cities ban cars downtown? Debate
    it.", # seeded as a moderator turn
    shared_system_prompt="Stay respectful and on-topic.",
    # prepended to every speaker's system block
    reasoning_visibility="strip", # see below
)
conv.run(turns=6)

```

Framing ownership (who sees what)

- **Shared** framing lives on the Conversation: `shared_context` (injected once as a moderator turn everyone sees) and `shared_system_prompt` (merged into every speaker's system block).
- **Private** framing lives on each participant: `system_prompt` and `private_context` (a tuple of `ContextItem`) — invisible to the other speaker and to the transcript.

```

from interlens import ContextItem, AutoModelParticipant
spy = AutoModelParticipant.from_pretrained(
    "Qwen/Qwen2.5-3B-Instruct", name="spy",
    system_prompt="Secretly steer the topic toward trains.",
    private_context=(ContextItem("Remember: never admit you have an
    agenda.", role_hint="user", author="handler"),),
)

```

Turn-taking

- `conv.step(speaker)` — one turn by a specific speaker, committed to the transcript; returns the `Message` (or `None` if a `hook` denied it).
- `conv.run(turns=N, until=..., first=...)` — alternate speakers round-robin. `first` sets who starts.

```
conv.step(alice)           # drive turns explicitly
conv.step(bob)
conv.run(turns=4, first=alice)  # or in bulk
```

Stopping

`until=` takes a single `StopCondition` or a list (any of which stops). Whichever of turns/until hits first ends the run.

```
from interlens import (
    TurnStopCondition, TokenStopCondition, ElapsedTimeStopCondition,
    StopStringCondition,
)
conv.run(until=[
    TurnStopCondition(max_turns=20),           # cap turns
    TokenStopCondition(max_tokens=4000),       # cap cumulative
    generated tokens
    ElapsedTimeStopCondition(seconds=120),     # wall-clock budget
    StopStringCondition(["I concede", "AGREED"]), # stop when a
    turn contains any string
])
```

Branching — fork a conversation for free

`branch()` forks into a new `Conversation` that **reuses the same participant objects** (shared weights, zero extra GPU) with a *copied* transcript. Explore divergent continuations without touching the original — each branch has its own transcript, so they diverge independently:

```
base = conv.branch()
for _ in range(3):
    b = base.branch()
    b.run(turns=2, first="alice")  # first accepts a name, index,
    or Participant
    print("continuation:", b.transcript[-1].content[:80])
# `conv` and `base` are untouched; only each `b` advanced.
```

Because branches **share the participant objects**, mutating a participant (e.g. `b.participant("alice").temperature = 0.3`) changes it for *every* branch and the original. To vary generation settings per branch, set them right before you run, or pin seed and vary only the prompt/intervention — see 09.

Fork from a specific turn — `branch_from`

`branch_from(ref)` forks as if the conversation had stopped **right after** `ref`, ready for a different continuation. `ref` is a *message reference*: an int index (Python semantics — negatives count from the end) or the `Message` object itself (matched by identity). The original is untouched.

```
alt = conv.branch_from(-3)           # continue as if the last two
    turns never happened
alt.run(turns=4, first="alice")
pivot = conv.transcript[2]           # a Message object
alt2 = conv.branch_from(pivot)        # fork right after that specific
    turn
```

Editing history in place — rewind, edit, reset

Sometimes you want to alter the *current* conversation rather than fork. These mutate in place (and return `self`/the `Message` for chaining); `branch_from` first if you'd rather keep the original.

```
conv.rewind(to=-2)                   # drop everything after that
    turn; `to` becomes the new last turn
conv.rewind(to=some_message)         # ...or rewind to a specific
    Message
conv.edit(3, "a corrected reply")     # replace turn 3's content
conv.edit(-1, author="moderator")     # change who a turn is
    attributed to
conv.edit(msg, note="flagged")        # merge metadata onto a turn
    (untouched keys survive)
```

`edit` targets are the same message references as `branch_from` (int index, negative, or `Message`). Because a `Message` is a mutable dataclass held by

reference, editing its fields directly — `conv.transcript[i].content = "..."`, or mutating the object `step()/sample()` returned — does the exact same thing; `edit` is just the ergonomic wrapper.

To wipe the dialogue and start the *same scenario* over, use `reset()` — it empties the transcript and re-seeds the `shared_context` framing:

```
conv.reset()                # back to the fresh, pre-run
    state (framing restored)
conv.run(turns=6)
```

`conv.reset()` keeps your `shared_context` / opening instructions; the lower-level `conv.transcript.clear()` does **not** — it wipes the seed turn too, leaving a genuinely empty transcript. Reach for `clear()` only when you want nothing at all left.

Editing history and the KV cache

Editing or rewinding is always safe with cross-turn KV reuse. The cache is reused only when the cached tokens are an **exact prefix** of the next prompt (`ModelParticipant._maybe_reuse_cache`); an edit changes tokens mid-history, so the prefix check fails and the model does a clean full prefill — the cache can never serve stale keys. A `rewind/branch_from` leaves a genuine prefix, so reuse still engages and only the new suffix is prefilled.

Ephemeral sampling — read state without mutating it

`sample()` generates a reply to an optional temporary message **without committing anything** — ideal for probing “what would X say now?” repeatedly.

```
for q in ["Summarize your position.", "What's your strongest
    objection?"]:
    print(conv.sample("alice", q, as_author="interviewer").content)
# conv.transcript is unchanged after all of this
```

`sample` and `step` accept the same interpretability options (`steering=`, `capture=`, `patch=`, `return_logprobs=`) — see [07](#).

Reasoning visibility (CoT models)

For models that emit `<think>...</think>`, `reasoning_visibility` controls whether a prior turn's parsed reasoning is re-injected into other speakers' views:

- "strip" (default) — reasoning never leaks into anyone's view.
- "self_retain" — a speaker sees only its *own* past reasoning.
- "shared" — everyone sees everyone's reasoning.

```
from interlens import ReasoningVisibility
conv = Conversation(participants=(alice, bob),
                    reasoning_visibility=ReasoningVisibility.SELF_RETAIN)
```

The raw completion and parsed reasoning are always stored per message in `msg.metadata["raw_completion"]` / `msg.metadata["parsed_think"]`.

Next: [participant & model options](#).